Introduction:

HTTP(Hyper Text Transfer protocol):
is an application-layer protocol used to exchange web requests and responses between a client and a server. A standard HTTP request uses TCP port 80 and does not provide encryption, which means the transmitted data can be rad by anyone who intercepts the traffic.

HTTPS URL Breakdown:

```
http://cybersecurity.com
```

Component Description

---

http Application-Layer Protocol
;// Separaotr
cybersecurity.com Domain name

we can send get request on the by using the http get request to access to any web page like as i send this request http://cybersecurity.com

---

HTTP GET Request Flow (Client Side) Encapsulation:

```
User types:
https://google.com
|
Browser
|
DNS Resolution
|
IP Address Found
|
Application Layer (HTTP)
creates HTTP Get Request
|
Transport layer (TCP)
Adds TCP Header
```

```
Creates TCP Segments
|
Network Layer (IP)
Creates IP Packet
|

Data Link Layer (Etehrnet)
Adds Ethernet Header
Creates Etehrnet Frame
|
Physical Layer
converts bits into electrical /
optical / wireless signals
|
Internet / ISP / Routers
|
Destination Servet
```

HTTP GET Request Flow (Server Side) Decapsulation:

```
Physical Layer
Receives Signals
|
Data lInk Layer
Removes Ethernet Header
|
Netwrok Layer
Removes IP header
|
Transport Layer
Removes TCP Header
Reassembles Segments
|
Application Layer
Process HTTP get request
|
Web server
Finds Requested page
|
Creates HTTP Response
```

Response Back to Client:

```
HTTP Response
|
Application Layer
|
Transport Layer (TCP)
|
Network Layer (IP)
|
Data Link Layer(Ethernet)
|
Physical Layer
|
Internet
|
client Physical Layer
|
DAta Link Layer
|
Network layer
|
Transport layer
|
Application Layer
|
Browser Displays Web Page
```

DNS Resolution

the browser first performs DNS resolution to obtain the server's IP address

```
User
|
Browser
|
DNS Cache
|
Operating System Cache
|
Router Cache
|
Recursive DNS Resolver
```

```
|
Root DNS Server
|TLD Server(.com)
|
Authoritative DNS Server
|
IP Address
```

he browser accepts a domain name from the user, but it must perform DNS resolution to obtain the server's IP addrss before establishing a connection of the server it needs ip sddress so it go to the dns server that check the about that weather it is .com .org or any thing else as google has .com so it go to .com server and than dns server gave the ip address sometime it an save in the DNS cache so that when we search about the same domain than it will get the ip address directly from that it saves time after getting the domain name .

Encapsulation:

flow chart:

```
Application Layer
|
Transport Layer (TCP)
|
Network Layer (IP)
|
Data Link Layer (Ethernet)
|
Physical Layer
```

Application Layer:

in an application-layer protocol used to exchange web requests and responses between a client and a server. A standard HTTP request uses TCP port 80 and does not provide encryption, which means the transmitted data can be read by anyone who intercepts the traffic. When a user enters a URL such as http://cybersecurity.com the browser first performs DNS resolution to obtain the server's IP addrss. The application layer creates the HTTP requesta as a sequence of bytes.

Transport Layer:

The data from the application layer is passed to the transport layer. The transport layer used to TCP for standard HTTP and HTTPS communication and addres a TCP haeader containinig information scan as source port, desination port, sequence number, adn acknowledgment number.

Network Layer:

transport layer send the segments to the network layer. network layer is used to select the best path to send that now in the network layer these segments are converted to the packets network layer add the its header that contain ip address(The Ip header contain the source and the destination ip address which allows routers to send the packets by diffrent paths but correct destination) network layer use the Router that is layer three devise use the ip addresses to select the best path to send the packet.Packets may take differnt paths through the network. TCP is responsible for reordering segments and delivering the data to the application in the correct sequence. these all packets are send in the form of sequsnses then it send it to the data link layer.

Data Link Layer:

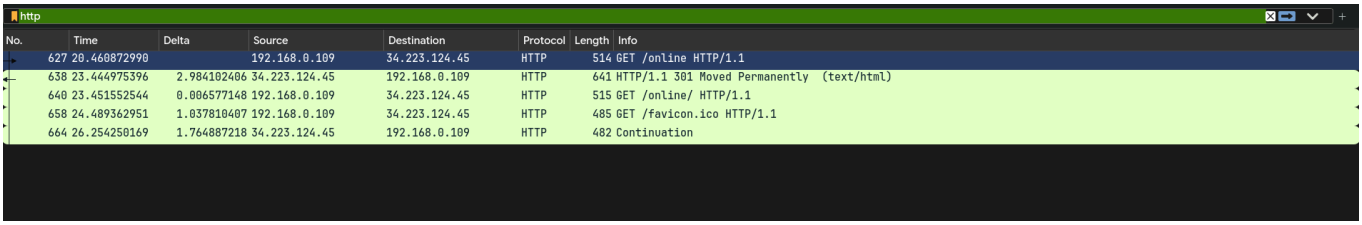
data link layer used physical address that we call mac addresses. These are used for local communication in the same network . The Data Link Layer ecapsulates the network-layer packet inside a frame.
The network layer then encapsulates the TCP segment int o an IP packet by adding the source and destination IP addrsses. The Data Link Layer encapsulates teh apcket into an Ethernet rame by adding source and destination IP addrsses. The Data Link Layer encapsulates the IP packet into Ethernet frame by adding source and destination MAC addresses.

Physical Layer:

Finally, the physical layer transmits the frame as electrical.optical,or wireless signals through the network. At the destination, the process is reversed throuth decapuslation until the applicaion layer receivers the original HTTP request.

Wireshark (capturing network traffic):

Packet List:



Captured packets:

- 627 GET /online HTTP/1.1
- 638 HTTP/1/1 301 Moved Permanently
- 640 GET /online HTTP/1.1
- 658 GET /favicon.ico HTTP/1.1
- 664 Continuation

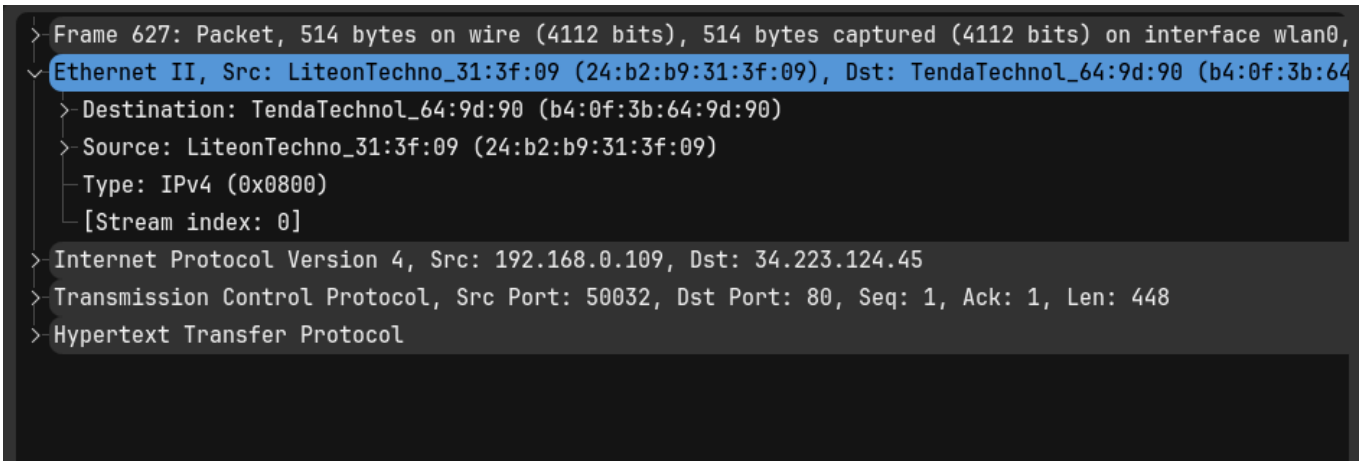
Wireshark Packet Overview Of Frame 627:

The captured packet (Frame 627) represents an HTTP GET request sent from my client machine (192.168.0.109) to the web server (34.223.124.45). Wireshark shows that this packet is encapsulated through multiple protocol layers. Starting from the application layer (HTTP), the data is encapsulated inside a TCP segment, tehn inside an IPv4 packet, and finally inside an Ethernet frame before being transmitted over the phusical network interface (wlan0).

The packet contains the following protocol layers:

- Frame - General information abour the captured packet.
- Etehrnet II - Data Link Layer information containing source and destination MAC addresses.
- Internet Protocol Version 4 (IPv4) - Network Layer information containing the source and destination IP addresses.
- Transmission Control Protocol (TCP) - Transport Layer information resposible for reliable communication.
- Hypertext Transfer Protocol(HTTP)- Application Layer protocol carrying teh HTTP GET request.

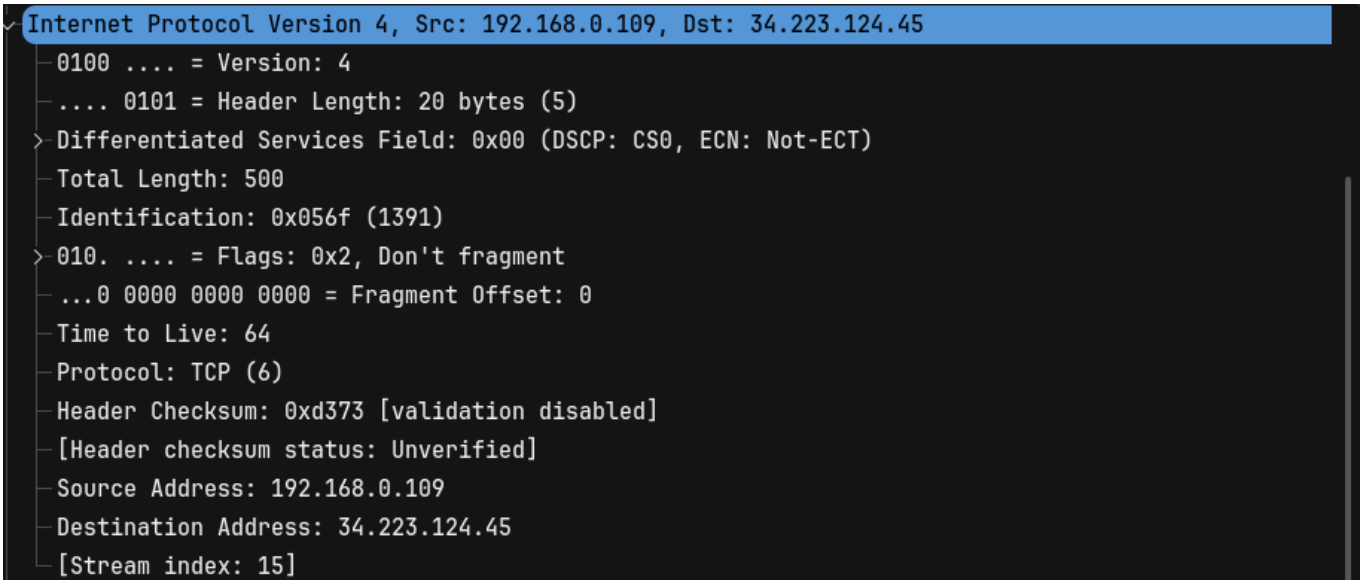
EthernetI Header Analysis:



The Ethernet II header is added by the Data Link Layer before the packet is transmitted on to the local network. It contains the source and destination MAC addresses, which identify devices on the local network segment. Unlike IP addresses, MAC addresses are only used for communication within the local network. The EtherType field (0x0800) indicates that the payload carried inside the Ethernet frame is an IPv4 packet

---

IPv4 Header Analysis



The IPv4 header is added by the Network Layer. Its main purpose is to provide logical addressing and routing information so that routers can forward packets across different networks.

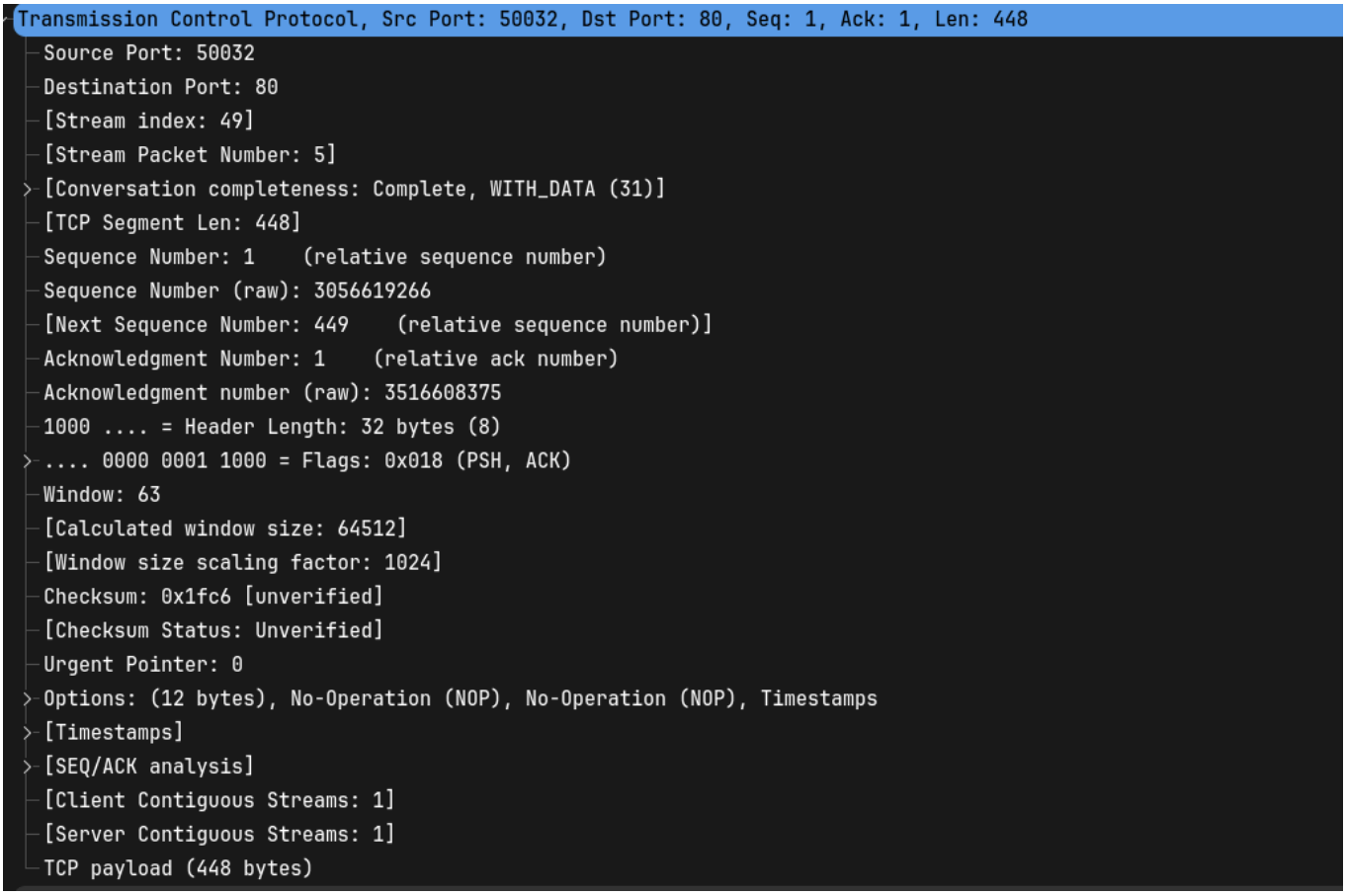
The IPv4 header contains:

- Version: 4 (specifies that the used IPV4)
- Header Length: 20 bytes (indicate where the IP payload begins)
- Total Length: 500 (show the complete size of the IP Packet)

- Identification: 0x056f (1391) (used if the packet must be fragmented)
- Flags: 0x2 (control Fragmentation)
- Fragment Offset: 0
- Time to Live: (TTL): 64 (prevent Packets from looping forever)
- Protocol: TCP (6) (Identifies the next protocol)
- Header Checksum: 0xd373 (check corruption in the IP header)
- source IP Address: 192.168.0.109
- Destination IP Address: 34.223.124.45

The source IP identifies my computer, while the destination IP identifies the web server. The TTL value decreases each time the packet passes through a router, preventing packets from circulating indefinitely. The Protocol field contains the value 6, which indicates that the payload is a TCP segment.

TCP Header Analysis:



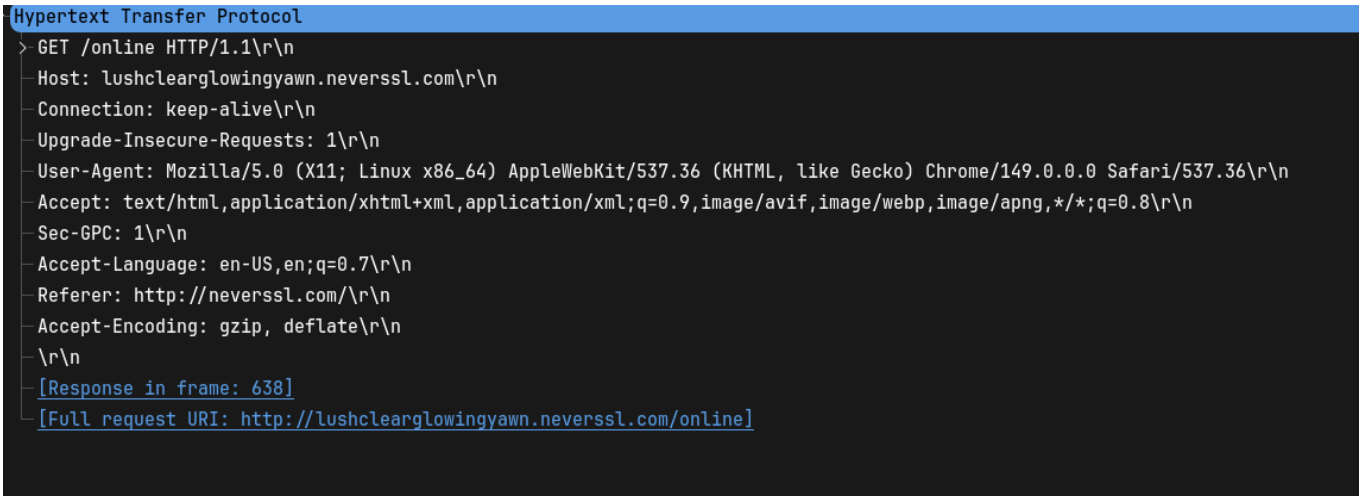
The TCP header is added at the Transport Layer. TCP provides reliable communication by ensuring that packets are delivered correctly, in order, and without data loss.

The captured TCP segment contains:

- Source Port: 50032 (Represents the temporary port opened by my browser)
- Destination Port: 80 (port 80 indicates the request is being sent to an HTTP server)
- Sequence Number: 1 (Keeps data in the correct order)
- Acknowledgment Number: 1 (confirm previously received bytes)
- Header Length: 32 bytes (8)
- Flags: 0x018 (PSH,ACK) (PSH tells the receiver to process the data immediately. ACK acknowledges previous packets)
- Window Size: 1024 (indicates how much data can be received before acknowledgment)
- checksum: 0x1fc6 (verifies integrity of the TCP segment)

The destination port 80 indicates that the request is sent to an HTTP web server . TCP uses sequence and acknowledgment numbers to grantee reliable delivery. The checksum is used to detect transmission errors.

HTTP Header Analysis:



The HTTP protocol operates at the Application Layer.
The captured request is:
Get /online HTTP/1.1
GET (HTTP method requesting data)
"/online" (Requested resource)
HTTP/1.1 (protocol version)
This indicates that the client is requesting the "/online" resource from the web server.
The HTTP request also contains several inportant headers:

- Host
- User-agent
- Accept

- Accept-Encoding
- Accept-Language
- Connection

The Host header identifies the destination on website. The User-Agent identifies the browser used t send the request. The Accept headers specify the content types supported by the client, while the connection header indicates whether the TCP connection should remain open after the response.

Packet Bytes Mapping (manual byte-by-byte):

```
b4 0f 3b 64 9d 90 24 b2  b9 31 3f 09 08 00 45 00   ··;d··$· ·1?···E·
01 f4 05 6f 40 00 40 06  d3 73 c0 a8 00 6d 22 df   ···o@·@· ·s···m"·
7c 2d c3 70 00 50 b6 30  4f 02 d1 9b 2f 77 80 18   |-·p·P·0 O···/w··
00 3f 1f c6 00 00 01 01  08 0a f4 cc 15 15 87 fb   ·?····· ········
43 b2 47 45 54 20 2f 6f  6e 6c 69 6e 65 20 48 54   C·GET /o nline HT
54 50 2f 31 2e 31 0d 0a  48 6f 73 74 3a 20 6c 75   TP/1.1·· Host: lu
73 68 63 6c 65 61 72 67  6c 6f 77 69 6e 67 79 61   shclearg lowingya
77 6e 2e 6e 65 76 65 72  73 73 6c 2e 63 6f 6d 0d   wn.never ssl.com·
0a 43 6f 6e 6e 65 63 74  69 6f 6e 3a 20 6b 65 65   ·Connect ion: kee
70 2d 61 6c 69 76 65 0d  0a 55 70 67 72 61 64 65   p-alive· ·Upgrade
2d 49 6e 73 65 63 75 72  65 2d 52 65 71 75 65 73   -Insecur e-Reques
74 73 3a 20 31 0d 0a 55  73 65 72 2d 41 67 65 6e   ts: 1··U ser-Agen
74 3a 20 4d 6f 7a 69 6c  6c 61 2f 35 2e 30 20 28   t: Mozil la/5.0 (
58 31 31 3b 20 4c 69 6e  75 78 20 78 38 36 5f 36   X11; Lin ux x86_6
34 29 20 41 70 70 6c 65  57 65 62 4b 69 74 2f 35   4) Apple WebKit/5
33 37 2e 33 36 20 28 4b  48 54 4d 4c 2c 20 6c 69   37.36 (K HTML, li
6b 65 20 47 65 63 6b 6f  29 20 43 68 72 6f 6d 65   ke Gecko ) Chrome
2f 31 34 39 2e 30 2e 30  2e 30 20 53 61 66 61 72   /149.0.0 .0 Safar
69 2f 35 33 37 2e 33 36  0d 0a 41 63 63 65 70 74   i/537.36 ··Accept
3a 20 74 65 78 74 2f 68  74 6d 6c 2c 61 70 70 6c   : text/h tml,appl
69 63 61 74 69 6f 6e 2f  78 68 74 6d 6c 2b 78 6d   ication/ xhtml+xm
6c 2c 61 70 70 6c 69 63  61 74 69 6f 6e 2f 78 6d   l,applic ation/xm
6c 3b 71 3d 30 2e 39 2c  69 6d 61 67 65 2f 61 76   l;q=0.9, image/av
69 66 2c 69 6d 61 67 65  2f 77 65 62 70 2c 69 6d   if,image /webp,im
61 67 65 2f 61 70 6e 67  2c 2a 2f 2a 3b 71 3d 30   age/apng ,*/*;q=0
2e 38 0d 0a 53 65 63 2d  47 50 43 3a 20 31 0d 0a   .8··Sec- GPC: 1··
41 63 63 65 70 74 2d 4c  61 6e 67 75 61 67 65 3a   Accept-L anguage:
20 65 6e 2d 55 53 2c 65  6e 3b 71 3d 30 2e 37 0d    en-US,e n;q=0.7·
0a 52 65 66 65 72 65 72  3a 20 68 74 74 70 3a 2f   ·Referer : http:/
2f 6e 65 76 65 72 73 73  6c 2e 63 6f 6d 2f 0d 0a   /neverss l.com/··
41 63 63 65 70 74 2d 45  6e 63 6f 64 69 6e 67 3a   Accept-E ncoding:
20 67 7a 69 70 2c 20 64  65 66 6c 61 74 65 0d 0a    gzip, d eflate··
0d 0a                                              ··
```

Byte Offset Hex Bytes Meaning

0-5 b4 0f 3b 64 9d 90 Destination MAC Address
6-11 24 b2 b9 31 3f 09 Source MAC Address
12-13 08 00 EtherType = IPv4
14 45 Version (4) + IHL (5=20 Bytes)
15 00 DSCP/ECN
16-17 01 f4 Total Length
18-19 05 6f Identification
20-21 40 00 Flags + Fragment Offset
22 40 TTL = 64
23 06 Protocol = TCP
24-25 d3 73 Header Checksum
26-29 c0 a8 00 6d Source IP Address
30-33 22 df 7c 2d Destination IP Address
34-35 c3 70 Source port
36-37 00 50 Destination Port
38-41 00 00 00 01 Acknowledgemet Number
46 80 header Length
47 18 Flags
48-49 04 00 window Size
50-51 1f c6 checksum
52-53 00 00 Urgent Pointer

The Packet bytes shown in Wireshark represent the raw hexadecimal data transmitted over the network. Each group of bytes corresponds to a specific field in the Ethernet, IPv4, TCP or HTTP headers. By manually mapping these bytes to their respective headers fields, it is possible to understand exactly how a browser request is encapsulated before transmission.

HTTP GET Request Flow Chart:

```
Application Layer
     |
HTTP GET Request
     |
TCP Header Added
     |
IPv4 Header Added
```

```
     |
Ethernet Header Added
     |
Physical Transmission
     |
Destiantion
```

Process Memory Address Space:

The browser process (brave/chrome) occupies a virtual memory space provided by operating system.
During the HTTP GET request the process uses:

- Text Segment
  Executable browser code
- data Segment
  Global variables
- Heap
  Stores dynamically allocated HTTP request objects, sockets buffer
- Stack
  Stores function calls and local variables while constructing the HTTP request.

The browser itself does not directly access physical memory .It works within its own virtual address space, which is translated by the operating system's memory management unit (MMU)

Step 1;
When i typed a URL.

```
http://google.com
```

Browser Process in RAM.

```
RAM
+-----------------------------+
| Chrome Process              |
+-----------------------------+
```

---

step 2:
Browser stores the URL temporary in the memory

```
google.com
```

this is stored in the temporary memory

---

step 3:
Browser call the DNS resolver function to obtain the server's IP address

```
getaddrinfo("google.com)
```

Step 4:
Browser creates s TCP socket before establishing the connection

```
socket()
connect()
send()
recb()
```

Information of socket also remain in the memory

Step 5:
Browser make the HTTP request.

```
GET / HTTP/1.1
Host: google.com
```

Step 6:
Operating System:
The operating system passes the HTTP request to the kernel's TCP/IP networking stack.

```
Browser
|
Operating System
|
TCP/IP Stack
|
NIC
|
Internet
```
